HUMAN–COMPUTER INTERACTION

Lab Midterm Assignment

Phase 3: Design

Polyline Editor — Interaction Design, UI Layout & Prototype Planning

Subject	Human–Computer Interaction
Phase	3 — Design
Reference	Dix et al., Human–Computer Interaction, 3rd Ed.
Prepared By	Neha Rehan-B23110006131

1. Introduction

The design phase bridges the analytical work of Phase 2 with implementation, focusing on how the system looks, behaves, and guides users effectively. In line with Dix et al. (2004), HCI design is grounded in understanding users' mental models, task structures, and interaction paradigms rather than aesthetics alone. For a spatial and gestural tool like a polyline editor, design decisions are driven by user needs, HCI principles, and potential risks. This document presents the interaction design, UI layout, task flows, HCI rationale, and prototype planning for the editor.

2. Interaction Design

2.1 Chosen Interaction Paradigm

The polyline editor adopts a mode-based direct manipulation paradigm. Direct manipulation, as defined by Shneiderman (1983) and discussed extensively by Dix et al. (2004, Chapter 4), involves the continuous representation of objects of interest, physical actions or labelled button presses instead of complex syntax, and rapid, incremental, reversible operations whose effect is immediately visible. All three properties are directly applicable to this tool:

- Points and line segments are always visible on the canvas.
- Users click or drag directly on graphical objects rather than issuing textual commands.
- Every action produces immediate visual feedback, the canvas redraws in real time.

The mode-based structure (Draw, Move, Delete, Refresh, Quit) is a deliberate design choice to prevent action ambiguity. In HCI literature, modeless designs can lead to mode errors when a single gesture carries multiple possible meanings depending on system state. By making the current mode explicit via a persistent status indicator, the system removes this ambiguity and reduces the cognitive load on the user.

2.2 Mode-to-Action Mapping

The table below maps each application mode to its triggering input and the resulting system action:

Mode	Trigger	User Action	System Response
Draw	Key: B / Button: Draw	Left-click on canvas	Adds vertex; line drawn from last point; press B again to finish current polyline and start a new one
Move	Key: M / Button: Move	Click nearest point, drag to new position	Point repositions; polyline redraws live

Delete	Key: D/Button:Delete	Click nearest point	Point removed; segments re-joined or line split
Refresh	Key: R / Button: Refresh	Press R or click Refresh	Canvas flashes white; all polylines again shows
Undo	Ctrl + Z / Button: Undo	Press Ctrl + Z or click Undo	Last action reverted (point added/removed or moved)
Redo	Ctrl + Y / Button: Redo	Press Ctrl + Y or click Redo	Last undone action reapplied
Clear	Ctrl + Delete / Button: Clear	Press Ctrl + Delete or click Clear	Canvas cleared; status message shown
Export PNG	Ctrl + S / Button: PNG	Press Ctrl + S or click PNG	Current canvas exported as PNG

Notes:

- Users can operate **via keyboard shortcuts or buttons** interchangeably.
- Style options are available for **line color**, **line width**, **point radius**, and **point color**.
- Undo/Redo support all actions: adding, deleting, and moving points.
- Refresh is different from Clear: it visually flashes the canvas to signal a full reset.

3. UI Layout & Wireframe Description

3.1 Layout Overview

The interface is intentionally minimal. Unnecessary chrome, toolbars, menus, panels competes with the user's attention and obscures the drawing canvas, which is the primary interaction surface. The layout is divided into three functional zones:

UI Zone	Purpose & HCI Rationale
Top Bar (Mode Indicator)	Displays the current active mode (e.g., 'MODE: DRAW') in high-contrast background color Dix et al. (2004) identify visibility of system status as a core usability requirement; this bar ensures the user always knows where they are.
Main Canvas	The dominant central region where polylines are drawn and manipulated. Points are rendered as filled

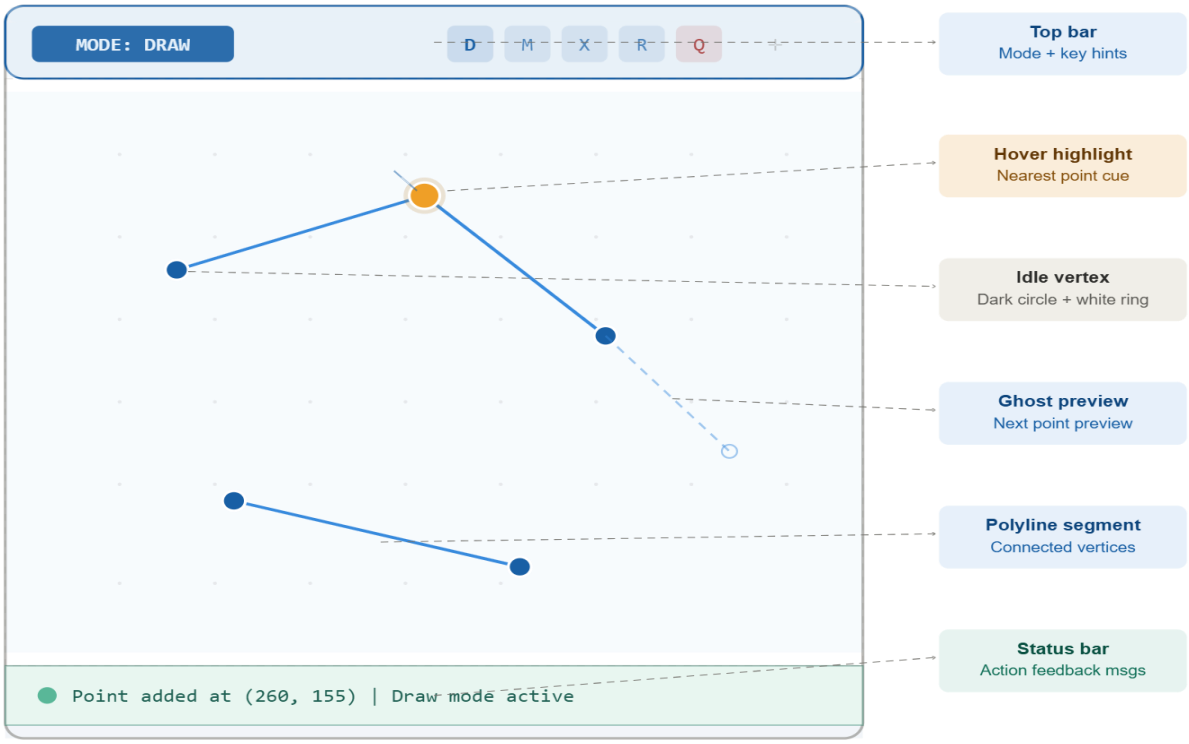
	circles; active/selected points are highlighted in a distinct colour (e.g., blue) to signal interactability.
Bottom Status Bar	Displays contextual feedback messages (e.g., 'Point moved successfully', 'Canvas refreshed'). This satisfies the feedback principle, the system always acknowledges user actions within an appropriate time frame (Norman, 1988, as cited in Dix et al., 2004).

3.2 Wireframe

Wireframe Diagram

```
+-------------------------------------------------------------+
|  MODE: DRAW                          [B] [M] [X] [R] [Q]    |<- Top Bar
|  (High-contrast, always visible)         (key hints)        |
+-------------------------------------------------------------+
|                                                             |
|       P1 (●)-----------(●) P2                               |
|                          \                                  |
|                           \                                 |  <- Canvas
|                            (●) P3                           |
|                                                             |
|                    (○) <- hover highlight on nearest point  |
|                                                             |
|  [cursor shows crosshair in Draw; pointer in Move/Delete]   |
+-------------------------------------------------------------+
|  Status: Point added at (345, 210)                          |<- Status Bar
+-------------------------------------------------------------+

● = rendered vertex (filled circle, dark grey)
○ = highlighted/nearest point (bright colour, changes on hover)
P1, P2, P3 = vertex labels
```



4. Task Flows

Task flows detail the step-by-step interaction sequence for each core operation. They are grounded in the user scenarios established in Phase 1 and are used here to validate that the interaction design satisfies those scenarios without introducing unnecessary steps. Task flows are derived from task analysis, and understanding the structure of a task reveals where the interface can either support or obstruct the user.

4.1 Task Flow: Drawing a Polyline

Task Flow — Draw Mode

Precondition: Application is running; no mode selected.

Step 1 → User presses 'B'
System highlights mode indicator: 'MODE: DRAW'
Status bar shows: 'Draw mode active. Click to add points.'

Step 2 → User left-clicks on canvas at position P1
System renders a filled circle at P1
Status bar shows: 'Point added at (x, y)'

Step 3 → User left-clicks at position P2
System renders filled circle at P2
System draws a line segment from P1 → P2

Step N → User continues clicking; each click extends the polyline

End → User switches mode (e.g., presses 'M') to stop drawing
Current polyline is preserved on canvas

4.2 Task Flow: Moving a Point

Task Flow — Move Mode

Precondition: At least one polyline with 2+ points exists on canvas.

Step 1 → User presses 'M'
Mode indicator updates: 'MODE: MOVE'
System activates hover highlight for nearest point

Step 2 → User moves mouse near an existing point P
System highlights P in hover colour (nearest-point algorithm)
Provides visual affordance: 'this point is selectable'

Step 3 → User clicks and holds on P
P changes to 'selected' colour
Status bar: 'Point selected. Drag to move.'

Step 4 → User drags mouse to new position P'
Polyline redraws in real time, following cursor
(Live feedback — Dix et al., 2004: continuous feedback during manipulation)

Step 5 → User releases mouse button
P is repositioned to P'
Status bar: 'Point moved to (x, y)'

4.3 Task Flow: Deleting a Point

Task Flow — Delete Mode

Precondition: At least one polyline exists.

Step 1 → User presses 'ctrl + delete'
Mode indicator: 'MODE: DELETE'
Status bar: 'Delete mode active. Click a point to remove it.'

Step 2 → User hovers over a point; nearest point is highlighted

Step 3 → User clicks the highlighted point
Point is removed from point array
If 2+ points remain: adjacent segments re-join
If 1 point remains: isolated point persists (no segment)
If 0 points remain: polyline object is discarded
Canvas redraws; status: 'Point deleted.'

4.4 Task Flow: Refreshing the Canvas

Task Flow — Refresh

Step 1 → User presses 'R'

Step 2 → System triggers flash animation:

Step 3 → Status bar: Canvas refreshed,Mode resets to default

5. HCI Principles Applied in Design

The following section documents the HCI principles that directly shaped the design decisions above. Each principle is cited from Dix et al. (2004) and mapped to a specific design element, establishing a clear chain from theory to implementation.

5.1 Feedback

Feedback is the property whereby the system informs the user about what it has done in response to an action. They note that a lack of feedback leads to user uncertainty, repeated actions, and errors. In this design, feedback is implemented at three levels:

- Immediate visual feedback: The canvas redraws on every mouse event during Move, so the user sees the effect of their action in real time.
- Confirmatory feedback: The status bar echoes the outcome of every discrete action ('Point added at...', 'Point deleted.', etc.).
- State-change feedback: The flash animation on Refresh provides salient, unmistakable confirmation that a system-level event has occurred.

5.2 Visibility of System Status

Nielsen's first usability heuristic,holds that the system should always keep users informed about what is going on. The persistent mode indicator in the top bar directly addresses this. At no point should a user be unsure whether they are in Draw mode or Move mode; the indicator removes this ambiguity entirely.

5.3 Direct Manipulation

The concept of direct manipulation, formalised by Shneiderman (1983), describes interaction where the user acts directly on visible representations of objects rather than through intermediary commands. In the polyline editor, the user clicks on points, drags them, and sees the polyline reshape, there is no command syntax to learn. This reduces the gulf of execution by minimising the distance between the user's intention and the available action.

5.4 Consistency

Consistency Is one of the most important properties of a usable interface: the same operation should work the same way in every context. In this editor, the interaction for both Move and Delete follows the same pattern, hover to highlight, click to act. This structural consistency means that once a user has learned one mode interaction, they can transfer that knowledge to the other, reducing the learning overhead.

5.5 Error Prevention and Forgiveness

There are two complementary strategies for handling errors: prevention (stopping errors before they occur) and tolerance (making errors easy to recover from). The polyline editor employs both.

Error Prevention: The Proximity Threshold

The nearest-point selection algorithm only returns a match if the cursor is within a defined threshold radius (e.g., 10px). Clicks outside this radius produce no action and a gentle status message. This prevents the most common accidental errors: selecting the wrong point because the click landed slightly off-target, or accidentally triggering a Move or Delete action while the user was intending to do something else.

Error Tolerance: Undo/Redo Stack

The most consequential irreversible action in the editor is Delete. A user who accidentally deletes the wrong point has no recourse without undo. As a form of error tolerance, the system does not punish mistakes, it allows them to be walked back. A Ctrl+Z undo stack will be implemented, storing the last N operations in memory. This is not merely a convenience; it is a fundamental safety mechanism that allows users to explore the interface with confidence.

5.6 Affordances

Affordances communicate the possible interactions available to the user through visual design alone. Note that perceived affordances are what drive intuitive interaction, users do not need documentation if the interface clearly signals what is interactive. In this design, hover highlights on nearby points serve as a real-time affordance signal: the point visually 'invites' interaction when the cursor approaches, without any label or instruction.

6. Design Alternatives and Trade-offs

Dix et al. are explicit: 'One of the most important things about design is trade-off, choosing which goals or constraints can be relaxed so that others can be met' (Dix et al., 2004, Ch. 5). This section documents the alternatives that were considered and the reasoning behind each choice. This is where design becomes genuine intellectual work rather than implementation.

6.1 Mode-Based vs. Tool-Based Interaction

Approach	Advantages	Disadvantages	Decision
Mode-based (chosen)	Simple mental model; consistent gesture across actions; minimal UI clutter	User must remember current mode; potential for wrong-mode errors	Chosen: simpler cognitive model for target users
Tool-based (e.g. Figma)	Mode always visible in cursor shape; no mental tracking	Requires a rich toolbar; more visual complexity; harder to implement consistently	Rejected:overcomplicated for a keyboard-first tool

Trade-off accepted: mode-based design trades away the convenience of always-visible tool state (partially recovered by the mode badge and toolbar) in exchange for a vastly simpler implementation and a more consistent interaction grammar.

6.2 Nearest-Point Threshold: Fixed vs. Dynamic

Approach	Advantages	Disadvantages	Decision
Fixed threshold, 10px (chosen)	Predictable; easy to reason about; consistent feel	May fail on dense polylines where points are <10px apart	Chosen: covers all normal use cases
Dynamic threshold (zoom-aware)	Adapts to point density; safer for complex diagrams	More complex; threshold changes can confuse muscle memory	Rejected: over-engineering for scope
Click-area highlight on hover	User sees exactly what will be selected before clicking	Requires additional rendering pass each frame	Incorporated as a supplement hover glow added

6.3 Feedback Style: Modal Dialog vs. Status Bar

Approach	Advantages	Disadvantages	Decision
Modal dialog for confirmations	Unambiguous; forces acknowledgement	Interrupts flow; frustrating for expert users; violates direct manipulation feel	Used only for Quit with unsaved changes
Status bar messages (chosen)	Non-intrusive; does not break drawing flow; fades cleanly	Can be missed if user is not watching the bottom of screen	Chosen:appropriate for low-stakes feedback
Toast notifications	More visually prominent; harder to miss	Can overlap canvas; adds animation complexity	Rejected:status bar is sufficient

7. Key Design Decisions & Rationale

Design Decision	Justification	HCI Reference
Mode-based structure	Prevents action ambiguity in a multi-operation tool	A. F. Monk. Mode errors: mode errors arise when gestures carry multiple meanings
Persistent mode indicator	User always knows current system state	Visibility of system status (Nielsen, 1994; Dix et al., 2004, Ch. 9,p. 325)
Hover highlight on nearest point	Signals interactable objects; reduces mis-clicks	Affordances (Dix et al., 2004, p. 217); error prevention
Real-time drag feedback	Immediate response during manipulation reduces uncertainty	Direct manipulation principle (Dix et al., 2004, p. 171)

Nearest-point threshold	Prevents accidental point selection in dense polylines	Error prevention (Dix et al., 2004, p. 283)
Keyboard for modes, mouse for space	Ergonomic task division; reduces hand-switching	Ergonomics (Dix et al., 2004, p. 131)
Undo / redo stack (planned)	Allows recovery from accidental deletions or moves	Reversibility and error forgiveness (Dix et al., 2004, p. 171)

8. Prototype Planning

8.1 Prototype Fidelity

For this project, a medium-fidelity interactive prototype will be developed using HTML5 Canvas, CSS, and vanilla JavaScript.

According to iterative design principles (Dix et al., 2004, chapter 6, p.241), requirements cannot be fully specified at the beginning; therefore, prototypes are used to explore and validate design decisions with real users. In this context, the prototype serves primarily an exploratory role, allowing early testing of interaction techniques before committing to full implementation.

A low-fidelity prototype such as a storyboard(p. 244) or paper sketch can represent layout and static interface elements, but it lacks the ability to demonstrate dynamic interaction, which is essential for this system. Features such as:

- real-time drawing on canvas
- hover-based highlighting
- drag-and-drop movement
- temporal feedback (e.g., flash animation)

require interactive simulation, which aligns with the book's concept of limited functionality prototypes.

The prototype will therefore include core interaction behavior while deliberately excluding non-essential features such as data persistence and file handling. This reflects a common trade-off in prototyping where functionality is prioritized over completeness, while some non-functional aspects (e.g., performance or robustness) may be simplified.

8.2 Prototype Scope

The prototype follows an incremental and simulation-based approach, where key interaction components are implemented and evaluated.

Feature	Behaviour
Canvas rendering	Points as filled circles; line segments between vertices
Mode switching	Buttons switch active mode; status bar updates accordingly
Draw mode	Left-click adds vertex; polyline grows with each click
Move mode	Hover highlights nearest point; click-drag repositions with live redraw
Delete mode	Hover highlights nearest point; click removes it and redraws
Refresh	Flash animation + canvas clear + status message
Style Options	Left sidebar controls for line color, width, point radius, point color
Undo / Redo	Single-step undo implemented for scenario testing
NOT included	File save/load, multi-polyline management (planned for implementation)

This scope reflects the idea of prototypes focusing on selected aspects of functionality, rather than the complete system.

8.3 Scenario Walkthrough Plan

The three user personas established in Phase 1 — Sara, Ahmad, and Priya — each represent a distinct usage pattern and a distinct failure mode. The prototype walkthrough is structured around their scenarios, not around feature checklists. This reflects the Phase 1 philosophy: design serves people, not functions.

Scenario 1 — Sara: Draw and Move

Sara draws a three-vertex polyline and moves the middle point.

- Start condition: Canvas is blank; mode is DRAW.
- Actions: Three clicks to create polyline; mode switch to MOVE; hover near middle point; drag.
- Expected: Hover ring appears as cursor approaches within 10px; drag tracks cursor in real time; all line segments redraw each frame; status bar confirms 'Point moved.'
- What we are validating: The 10px threshold, does it feel too tight (user misses the point) or too generous (wrong point captured)?
- What counts as failure: Any moment where Sara is uncertain whether her hover registered, or whether her drag is being tracked.

Scenario 2 — Ahmad: Accidental Deletion and Undo

Ahmad accidentally deletes the wrong point and needs to recover.

- Start condition: A four-vertex polyline is on canvas; mode is DELETE.
- Actions: Click slightly off-target, deleting the wrong point; attempt Ctrl+Z.
- Expected: Status bar shows 'Point deleted.' in red immediately; Ctrl+Z restores the point and redraws; status bar updates to 'Undo successful.'
- What we are validating: Whether the delete feedback is visible and immediate enough for Ahmad to notice before attempting further actions; whether the undo interaction is fast enough to feel like true reversal rather than reconstruction.
- What counts as failure: Ahmad does not notice the deletion happened, or does not know undo is available.

9. References

Dix, A., Finlay, J., Abowd, G. D., & Beale, R. (2004). Human–Computer Interaction (3rd ed.). Pearson Education Limited.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

Norman, D. A. (1988). The design of everyday things. Basic Books.

[237] A. F. Monk. Mode errors: a user-centred analysis and some preventative

measures using keying contingent sound. International Journal of Man–Machine

Studies, 24, 1986.